

Designing and Managing Data Lakes

Raw vs Curated Zones • Parquet • ORC • Partitioning Strategies

Session 4 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2024–2025

Session Overview

Duration	4 hours (theory + lab)
Theory	Data lake zones, storage formats, partitioning, governance
Lab	Ingest Kafka events into a structured data lake with Spark SQL
Prerequisites	Sessions 1–3 completed
Tools used	PySpark 3.5, Parquet, Delta Lake (optional), Docker

🎯 Learning Objectives

By the end of this session, students will be able to:

- ✔ Define a data lake and contrast it with a data warehouse and data lakehouse.
- ✔ Design a three-zone data lake architecture (raw, curated, consumption).
- ✔ Explain the internal structure of Parquet and ORC and choose between them.
- ✔ Apply partitioning strategies to optimise read performance and avoid small-file problems.
- ✔ Ingest Kafka sensor events into a partitioned Parquet data lake using PySpark.
- ✔ Query the data lake with Spark SQL using partition pruning.

Contents

1	Part I – Data Lakes: Concepts and Architecture	3
1.1	What Is a Data Lake?	3
1.2	Data Lake vs Data Warehouse vs Data Lakehouse	3
1.3	The Three-Zone Architecture	4
1.3.1	Zone 1: Raw Zone (Bronze / Landing)	4
1.3.2	Zone 2: Curated Zone (Silver / Standardised)	4
1.3.3	Zone 3: Consumption Zone (Gold / Serving)	4
1.4	Directory Naming Conventions	4
2	Part II – Storage Formats: Parquet, ORC, and Avro	6
2.1	Row-Oriented vs Column-Oriented Storage	6
2.2	Apache Parquet	6
2.2.1	Parquet Internal Structure	7
2.2.2	Parquet Key Features	7
2.3	Apache ORC	7
2.3.1	Parquet vs ORC: When to Choose	8
2.4	Apache Avro (Row-Oriented – for Kafka)	8
3	Part III – Partitioning Strategies	9
3.1	What Is Partitioning in a Data Lake?	9
3.2	Choosing Partition Columns	9
3.3	The Small File Problem	10
3.4	Partitioning for This Session’s Data Lake	10
4	Part IV – Data Lake Governance	11
4.1	Why Governance Matters	11
4.2	The Four Pillars of Data Lake Governance	11
4.3	Metadata File Convention for This Lab	11
5	Part V – Lab Session	13
5.1	Step 0 – Setup	13
5.1.1	0a – Create the Data Lake Directory Structure	13
5.1.2	0b – Ensure the Kafka Cluster and Topic Are Ready	13
5.2	Step 1 – Raw Zone: Land Data from Kafka	14
5.2.1	1a – Imports and SparkSession	14
5.2.2	1b – Define Schema and Read Raw Kafka Stream	14
5.2.3	1c – Write Raw Zone (JSON format, partitioned by date)	15
5.3	Step 2 – Curated Zone: Clean, Parse, and Partition	16
5.3.1	2a – Parse and Validate	16
5.3.2	2b – Add Partitioning Columns from Event Time	16
5.3.3	2c – Write Curated Zone (Parquet, Snappy compression)	16
5.3.4	2d – Start Both Queries and Wait	17
5.4	Step 3 – Consumption Zone: Build Aggregated Dataset	17
5.5	Step 4 – Query with Spark SQL and Partition Pruning	18
5.5.1	4a – Register the Curated Dataset as a Table	18

5.5.2	4b – Query 1: Basic Partition Pruning	18
5.5.3	4c – Query 2: Anomaly Analysis	19
5.5.4	4d – Measure the Pruning Benefit	19
6	Summary & Key Takeaways	21
6.1	Vocabulary Checklist	21
6.2	Further Reading	21
6.3	Preview: Session 5	22
Appendix A – Complete datalake_pipeline.py		23
Appendix B – Glossary		26

1. Part I – Data Lakes: Concepts and Architecture

1.1 What Is a Data Lake?

Definition – Data Lake

A **data lake** is a centralised repository that stores data in its *native, raw format* – structured, semi-structured, and unstructured – at any scale, using cheap commodity object storage (HDFS, Amazon S3, Azure Data Lake Storage, Google Cloud Storage). Unlike a data warehouse, a data lake imposes *no schema at write time*; schema is applied when data is read (*schema-on-read*).

The data lake concept was coined by James Dixon (Pentaho, 2010) to contrast with the *data mart* metaphor: a data mart stores a curated, bottled subset; a lake stores everything in its natural state.

1.2 Data Lake vs Data Warehouse vs Data Lakehouse

Table 1: Three storage paradigms compared

lightbg Dimension	Data Warehouse	Data Lake	Data Lakehouse
Schema enforcement	Write time (rigid)	Read time (flexible)	Read time + optional constraints
Data types	Structured only	All (raw, JSON, images...)	All
Storage cost	High (proprietary)	Very low (object store)	Low (object store)
Query performance	Excellent (pre-optimised)	Slow without tuning	Good (table format layer)
ACID transactions	Yes	No (by default)	Yes (Delta Lake, Iceberg)
Typical users	Analysts, BI tools	Data engineers, DS	All (unified)
Examples	Snowflake, Redshift	S3 + Parquet, HDFS	Delta Lake, Apache Iceberg

The Data Swamp Problem

A data lake without governance quickly becomes a **data swamp**: data is ingested but never catalogued, documented, or organised. No one knows what exists, in what format, or how fresh it is. The solution is a **three-zone architecture** with clear ingestion rules, naming conventions, and a metadata catalogue.

1.3 The Three-Zone Architecture

The most widely adopted data lake design separates storage into three logical zones, each with a distinct purpose and access policy.

1.3.1 Zone 1: Raw Zone (Bronze / Landing)

- Stores data **exactly as received** from the source – no transformation, no schema enforcement.
- **Immutable**: files are never overwritten or deleted (append-only). This is your audit trail and replay source.
- Format: whatever the source produces (JSON, CSV, Avro, binary logs, images).
- Organised by **ingestion time**: `raw/source=kafka/topic=sensor-events/year=2024/month=01/day=1`
- Access: restricted to data engineers and automated pipelines.

1.3.2 Zone 2: Curated Zone (Silver / Standardised)

- Data has been **validated, cleaned, deduplicated**, and converted to a standard columnar format (Parquet or ORC).
- Schema is enforced; nulls and type errors are handled.
- Organised by **business entity and event time**: `curated/domain=iot/sensor_type=temperature/year=2024`
- Access: data engineers and data scientists.

1.3.3 Zone 3: Consumption Zone (Gold / Serving)

- Contains **aggregated, enriched, business-ready** datasets built from curated data.
- Optimised for query performance: pre-joined wide tables, denormalised for BI tools.
- Access: all users – analysts, BI dashboards, ML pipelines.

1.4 Directory Naming Conventions

A consistent naming convention is essential for discoverability. The recommended pattern for this course is:

```
# Raw zone: organised by ingestion metadata
/datalake/raw/
  source=<system>/
  topic=<name>/
  year=<YYYY>/month=<MM>/day=<DD>/hour=<HH>/
  part-00000-<uuid>.json

# Curated zone: organised by business domain and event time
/datalake/curated/
  domain=<name>/
  entity=<type>/
  year=<YYYY>/month=<MM>/day=<DD>/
```

```
part-00000-<uuid>.parquet

# Consumption zone: organised by use-case
/datalake/consumption/
  use_case=<name>/
    report_date=<YYYY-MM-DD>/
      part-00000-<uuid>.parquet
```

Using **key=value** directory naming (e.g., `year=2024/month=01`) is called **Hive-style partitioning**. Spark, Hive, Athena, and Presto all recognise this pattern automatically and perform *partition pruning*: when a query filters on `year=2024`, Spark only reads directories matching that filter – skipping all other years entirely.

2. Part II – Storage Formats: Parquet, ORC, and Avro

2.1 Row-Oriented vs Column-Oriented Storage

The most important axis when choosing a file format is whether data is stored *row-by-row* or *column-by-column*.

Row-Oriented (CSV, JSON)			Column-Oriented (Parquet, ORC)		
sensor	value	unit	sensor	value	unit
temp	28.5	C	temp	28.5	C
hum	62.1	pct	hum	62.1	pct
temp	36.8	C	temp	36.8	C

Row 0 together

value coltogether

Figure 1: Row-oriented: each row is contiguous on disk. Column-oriented: each column is contiguous – perfect for analytical queries that read only a few columns from millions of rows.

Why column-oriented storage wins for analytics:

- **Column pruning:** a query reading only **value** and **sensor** skips the **unit**, **timestamp**, and **source** columns entirely – often 60–80% of bytes.
- **Better compression:** a column of the same type has much higher redundancy than a mixed row, so compression algorithms (Snappy, Zstd, LZ4) achieve 5–10× smaller files.
- **Vectorised execution:** CPUs can apply SIMD instructions to a block of 1024 identical-type values in a single operation.

2.2 Apache Parquet

Definition – Apache Parquet

Apache Parquet is an open-source, **columnar binary file format** designed for efficient analytical processing. It organises data into *row groups* (horizontal slices), within which each column is stored as *column chunks* with embedded statistics (**min**, **max**, **null_count**) that enable *predicate pushdown* to skip row groups that cannot satisfy a filter condition.

2.2.1 Parquet Internal Structure

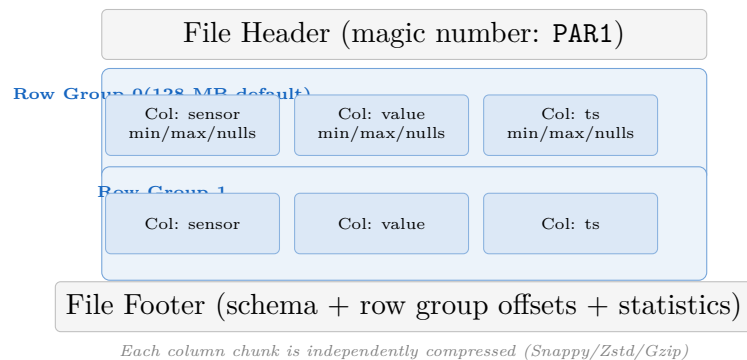


Figure 2: Parquet file layout: row groups contain column chunks; the footer holds schema and statistics for predicate pushdown.

2.2.2 Parquet Key Features

lightbg Feature	Description
Columnar storage	Only required columns are read from disk
Row group statistics	min/max/null_count per column chunk – enables predicate pushdown
Nested data support	Supports arrays, maps, and structs (Dremel encoding)
Schema evolution	Add new nullable columns without rewriting files
Language support	Spark, Hive, Pandas, DuckDB, Arrow, Athena, BigQuery
Compression codecs	Snappy (fast), Gzip (compact), Zstd (best ratio), LZ4 (balanced)

2.3 Apache ORC

Definition – ORC (Optimised Row Columnar)

ORC is a columnar file format developed by Hortonworks and optimised for Apache Hive workloads. Like Parquet, ORC uses columnar storage, compression, and predicate pushdown. ORC additionally includes **built-in indexes** (row-level, bloom filters) and **ACID transaction support** (with Hive 3+).

2.3.1 Parquet vs ORC: When to Choose

Table 2: Parquet vs ORC feature comparison

lightbg Dimension	Parquet	ORC
Ecosystem fit	Spark, Python (Arrow), Presto, Iceberg	Hive, HBase, Cloudera
Nested data	Excellent (Dremel encoding)	Good
Compression	Snappy default; Zstd excellent	Zlib default; very compact
Predicate pushdown	Row group min/max statistics	Row-level indexes + bloom filters
ACID transactions	Requires Delta Lake or Iceberg	Built-in (Hive ACID)
Query performance	Excellent with Spark	Excellent with Hive
Choose when	Spark, Arrow, modern cloud stack	Hive-centric Hadoop environment

👍 Course Recommendation

Use **Parquet** for all lab exercises and new projects. It has the broadest ecosystem support, excellent Spark integration, and is the default format for Delta Lake and Apache Iceberg. ORC is the right choice only when your stack is primarily Hive-based.

2.4 Apache Avro (Row-Oriented – for Kafka)

📖 Definition – Apache Avro

Avro is a **row-oriented** binary serialisation format with a schema stored alongside the data (in JSON). Unlike Parquet and ORC, Avro is optimised for *write-heavy*, *streaming* workloads where all fields of a record must be written together.

Table 3: Three-format summary

lightbg Format	Orientation	Best for	Common use
Parquet	Columnar	Analytical reads (many rows, few columns)	Data lakes, Spark, Athena
ORC	Columnar	Hive analytical workloads	Hadoop/Hive environments
Avro	Row	Streaming writes, Kafka, schema evolution	Kafka + Schema Registry

3. Part III – Partitioning Strategies

3.1 What Is Partitioning in a Data Lake?

Definition – Data Lake Partitioning

Partitioning organises files into a *directory hierarchy* based on one or more column values. When a query filters on a partitioned column, the query engine reads *only* the matching directories – skipping all others entirely. This is called **partition pruning** and is the single most impactful query optimisation technique in a data lake.

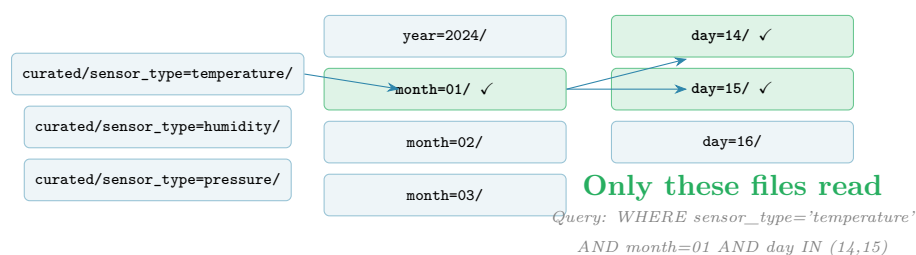


Figure 3: Partition pruning: only matching directories are opened. Months 02–03 and all other sensor types are completely skipped.

3.2 Choosing Partition Columns

The right partition columns depend on how data will be queried. Here are the key rules:

1. **Partition on frequently filtered columns.** If 90% of queries filter on `date` and `sensor_type`, partition on those two columns.
2. **Avoid high-cardinality columns.** Partitioning on `user_id` with 10 million users creates 10 million directories – metadata overhead kills performance. Rule of thumb: partition columns should have < 10,000 distinct values.
3. **Time partitioning is almost always correct.** `year / month / day` (or `year / month / day / hour` for high-volume streams) aligns with data retention policies and nearly all analytical queries filter on time ranges.
4. **Beware data skew.** If 80% of events have `sensor_type=temperature`, that partition will be much larger than others, creating *hot partitions*.
5. **Target Parquet file size: 128 MB – 1 GB.** Too-small files (“small file problem”) create excessive metadata and slow down both reads and writes. Too-large files limit parallelism.

3.3 The Small File Problem

⚠ Common Pitfall

Writing one Parquet file per micro-batch in Spark Streaming (e.g., every 10 seconds) creates **thousands of tiny files** over a day. HDFS and S3 have a metadata overhead per file: reading 10,000 files of 1 MB each is much slower than reading 100 files of 100 MB each, even though the total data is the same.

Solutions:

- **Compaction jobs:** a nightly Spark job reads the small files and rewrites them as larger ones.
- **Longer trigger intervals:** use `trigger(processingTime="1 hour")` instead of 10 seconds for non-latency-sensitive pipelines.
- **Delta Lake / Iceberg:** table formats with built-in compaction (`OPTIMIZE` command).
- **Coalesce before write:** `df.coalesce(N).write.parquet(...)` to control the number of output files.

3.4 Partitioning for This Session's Data Lake

For our IoT sensor dataset, the recommended partition scheme is:

```
# Curated zone partition hierarchy
curated/
  sensor_type=temperature/
    year=2024/
      month=01/
        day=15/
          part-00000-abc123.snappy.parquet
          part-00001-def456.snappy.parquet
  sensor_type=humidity/
    year=2024/ ...
  sensor_type=pressure/
    year=2024/ ...
```

Why sensor_type first? Most queries will filter on sensor type (“give me all temperature readings”), so putting it as the outermost partition eliminates irrelevant data immediately.

Why year/month/day? Analytical queries typically have date ranges. This hierarchy allows pruning at each level: a query for January 2024 reads only `year=2024/month=01/`.

4. Part IV – Data Lake Governance

4.1 Why Governance Matters

Without governance, a data lake devolves into a data swamp:

- No one knows *what datasets exist*.
- No one knows *who owns* a dataset or who to contact.
- No one knows *how fresh* data is or *what it means*.
- Sensitive data (PII, financial records) is mixed with public data.

4.2 The Four Pillars of Data Lake Governance

lightbg Pillar	What it means	Tools
Data Catalogue	Inventory of all datasets with metadata (schema, owner, freshness, tags)	AWS Glue, Apache Atlas, DataHub
Data Lineage	Track where each dataset came from and what downstream datasets depend on it	OpenLineage, Marquez, dbt lineage
Data Quality	Automated validation rules: non-null, range checks, referential integrity	Great Expectations, Soda, dbt tests
Access Control	Role-based and attribute-based permissions per zone and per dataset	Lake Formation (AWS), Unity Catalog, Ranger

4.3 Metadata File Convention for This Lab

For simplicity, we will store a `_metadata.json` file in each curated zone partition describing its contents:

```
{
  "dataset": "sensor-events-curated",
  "zone": "curated",
  "sensor_type": "temperature",
  "partition": "year=2024/month=01/day=15",
  "row_count": 1482,
  "schema_version": "1.0",
  "created_by": "etl_pipeline_v3",
  "created_at": "2024-01-15T23:00:00Z",
  "source_topic": "sensor-events",
  "source_offsets": {
    "0": 15000,

```

```
"1": 14800 ,  
"2": 15100  
}  
}
```

5. Part V – Lab Session

Lab 4 – Ingest and Organise Data in a Data Lake

Duration 90 minutes

Objective Build a three-zone data lake on the local filesystem: ingest raw Kafka events, curate v

Tools PySpark 3.5, Parquet, Docker Compose

Difficulty ●●● Intermediate

Lab Timeline

Time	Activity	Goal
0–10 min	Setup: directories + topic	Verify environment
10–25 min	Step 1: Raw zone ingestion	Land raw JSON from Kafka
25–45 min	Step 2: Curate to Parquet	Validate, partition, compress
45–60 min	Step 3: Build consumption zone	Windowed aggregation
60–75 min	Step 4: Spark SQL + partition pruning	Query the lake efficiently
75–90 min	Step 5: Measure pruning gain	Compare scan with/without filter

5.1 Step 0 – Setup

5.1.1 0a – Create the Data Lake Directory Structure

```
# Create the three-zone structure
mkdir -p /tmp/datalake/{raw,curated,consumption}
mkdir -p /tmp/datalake/raw/source=kafka/topic=sensor-events
mkdir -p /tmp/datalake/curated/domain=iot
mkdir -p /tmp/datalake/consumption/use_case=sensor_averages
echo "Structure created."
find /tmp/datalake -type d
```

We simulate a data lake on the local filesystem. In production, replace `/tmp/datalake` with an S3 URI (`s3://my-bucket/datalake/`) or an HDFS path (`hdfs://namenode:9000/datalake/`). The PySpark code is **identical** – only the root path changes.

5.1.2 0b – Ensure the Kafka Cluster and Topic Are Ready

```
docker compose ps          # in tp1_solution/
docker exec kafka1 kafka-topics \
  --bootstrap-server kafka1:29092 --list
```

The `sensor-events` topic must exist with at least some messages. Run `python producer.py -count 500` from TP3 to populate it before starting Spark. More messages means more meaningful statistics across multiple time windows and sensor types.

5.2 Step 1 – Raw Zone: Land Data from Kafka

Create `datalake_pipeline.py`. Build it section by section.

5.2.1 1a – Imports and SparkSession

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, from_json, to_timestamp,
    expr, year, month, dayofmonth,
    hour, lit
)
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType,
    LongType, BooleanType
)
import os
from datetime import datetime

# Paths
LAKE_ROOT      = "/tmp/datalake"
RAW_PATH       = f"{LAKE_ROOT}/raw/source=kafka/topic=sensor-events"
CURATED_PATH   = f"{LAKE_ROOT}/curated/domain=iot"
CONSUME_PATH   = f"{LAKE_ROOT}/consumption/use_case=sensor_averages"
CKPT_RAW       = "/tmp/datalake-ckpt/raw"
CKPT_CUR       = "/tmp/datalake-ckpt/curated"

KAFKA_BROKERS  = "localhost:9092,localhost:9094,localhost:9096"
TOPIC          = "sensor-events"

spark = SparkSession.builder \
    .appName("DataLakePipeline") \
    .master("local[*]") \
    .config(
        "spark.jars.packages",
        "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3"
    ) \
    .config("spark.sql.shuffle.partitions", "3") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

We define paths for all three zones upfront. Using constants avoids hard-coded strings spread throughout the code – if the root path changes, only one line needs updating.

5.2.2 1b – Define Schema and Read Raw Kafka Stream

```
SENSOR_SCHEMA = StructType([
    StructField("sensor", StringType(), False),
```



```

    StructField("value",      DoubleType(),  False),
    StructField("unit",       StringType(),   True),
    StructField("timestamp",  LongType(),     False),
    StructField("source",     StringType(),   True),
    StructField("anomaly",    BooleanType(),  True),
  ])

raw_kafka = spark.readStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
  .option("subscribe", TOPIC) \
  .option("startingOffsets", "earliest") \
  .option("failOnDataLoss", "false") \
  .load()

```

5.2.3 1c – Write Raw Zone (JSON format, partitioned by date)

```

# For the raw zone we keep data as JSON strings -- no parsing.
# This preserves the exact original payload for future replay.
raw_for_lake = raw_kafka.select(
  col("value").cast("string").alias("raw_json"),
  col("partition").alias("kafka_partition"),
  col("offset").alias("kafka_offset"),
  col("timestamp").alias("ingestion_ts"),
  # Extract date parts from INGESTION time for raw zone
  year(col("timestamp")).alias("year"),
  month(col("timestamp")).alias("month"),
  dayofmonth(col("timestamp")).alias("day"),
  hour(col("timestamp")).alias("hour"),
)

query_raw = raw_for_lake \
  .writeStream \
  .outputMode("append") \
  .format("json") \
  .option("path", RAW_PATH) \
  .option("checkpointLocation", CKPT_RAW) \
  .partitionBy("year", "month", "day", "hour") \
  .trigger(processingTime="30 seconds") \
  .start()

```

Raw zone uses ingestion time, not event time. The raw zone answers “when did this data arrive in our system?” – useful for debugging pipeline delays and SLA monitoring. The curated zone will use event time for business queries.

.partitionBy("year", "month", "day", "hour"): Spark writes files into nested directories: year=2024/month=01/day=15/hour=10/part-00000.json.

format("json"): raw zone preserves the original JSON strings. No schema is imposed – if the source format changes, old files remain readable.

5.3 Step 2 – Curated Zone: Clean, Parse, and Partition

5.3.1 2a – Parse and Validate

```
parsed = raw_kafka.select(
    from_json(
        col("value").cast("string"),
        SENSOR_SCHEMA
    ).alias("d"),
    col("partition"),
    col("offset"),
) \
.select(
    col("d.sensor").alias("sensor_type"),
    col("d.value"),
    col("d.unit"),
    col("d.anomaly").alias("is_anomaly"),
    to_timestamp(
        expr("d.timestamp / 1000")
    ).alias("event_time"),
    col("partition"),
    col("offset"),
) \
.filter(col("sensor_type").isNotNull()) \
.filter(col("value").isNotNull()) \
.filter(col("value").between(-100, 2000))
```

.filter(col("value").between(-100, 2000)): a range check is a basic data quality gate. Values outside this range are physically impossible for our sensors and indicate corrupt data. In production, rejected records go to a quarantine partition, not silently dropped. The curated zone uses **event time** (when the measurement happened) not ingestion time, because business queries ask “what was the average temperature on January 15?” – referring to the real-world time of the measurement.

5.3.2 2b – Add Partitioning Columns from Event Time

```
curated = parsed \
    .withColumn("year", year(col("event_time"))) \
    .withColumn("month", month(col("event_time"))) \
    .withColumn("day", dayofmonth(col("event_time")))
```

We add year, month, day as explicit columns derived from event_time. Spark’s partitionBy() needs column names, not expressions.

Important: the partitioning columns (sensor_type, year, month, day) are *not* stored inside the Parquet file – they are encoded in the directory path. Spark re-adds them as virtual columns when reading the partitioned dataset.

5.3.3 2c – Write Curated Zone (Parquet, Snappy compression)

```
query_curated = curated \
```

```

.writeStream \
.outputMode("append") \
.format("parquet") \
.option("path", CURATED_PATH) \
.option("checkpointLocation", CKPT_CUR) \
.option("compression", "snappy") \
.partitionBy("sensor_type", "year", "month", "day") \
.trigger(processingTime="30 seconds") \
.start()

```

compression="snappy": Snappy is the default and best all-around choice – fast compression and decompression, moderate compression ratio (2–3×). Use **zstd** when storage cost matters more than CPU cost (4–6× compression).

partitionBy("sensor_type", "year", "month", "day"): **sensor_type** is the outermost partition because the most common query pattern filters on sensor type first. Partition order determines directory nesting order.

5.3.4 2d – Start Both Queries and Wait

```

print("Data lake pipeline running. Ctrl+C to stop.")
print(f"  Raw zone      : {RAW_PATH}")
print(f"  Curated zone   : {CURATED_PATH}")

try:
    spark.streams.awaitAnyTermination()
except KeyboardInterrupt:
    print("Stopping all queries...")
    for q in spark.streams.active:
        q.stop()

```

spark.streams.awaitAnyTermination() blocks until any query terminates (error or manual stop). This is the correct pattern when running multiple parallel streaming queries on the same **SparkSession**.

5.4 Step 3 – Consumption Zone: Build Aggregated Dataset

After the streaming pipeline has written some data, run this **batch** job to build the consumption zone.

```

# Read the curated zone as a BATCH dataset
curated_df = spark.read \
    .format("parquet") \
    .load(CURATED_PATH)

print(f"Curated zone: {curated_df.count()} records")
curated_df.printSchema()

#           Aggregate: daily averages per sensor type

from pyspark.sql.functions import avg, min, max, count, sum

```

```

daily_agg = curated_df \
    .groupBy("sensor_type", "year", "month", "day") \
    .agg(
        count("value").alias("record_count"),
        avg("value").alias("avg_value"),
        min("value").alias("min_value"),
        max("value").alias("max_value"),
        sum(col("is_anomaly").cast("int")).alias("anomaly_count"),
    ) \
    .orderBy("sensor_type", "year", "month", "day")

# Write to consumption zone
daily_agg.write \
    .mode("overwrite") \
    .partitionBy("sensor_type", "year", "month") \
    .parquet(CONSUME_PATH)

print("Consumption zone written.")
daily_agg.show(20, truncate=False)

```

The consumption zone is built as a **batch** job (not streaming) because it reads from the already-curated data. This is the Gold layer in the Medallion architecture.

.mode("overwrite"): the consumption zone is *recomputed* from curated data on each run – it is always reproducible and does not need a checkpoint. This is the key advantage of ELT: you can always re-derive downstream tables from upstream raw/curated data.

5.5 Step 4 – Query with Spark SQL and Partition Pruning

5.5.1 4a – Register the Curated Dataset as a Table

```

# Read the curated Parquet dataset and register as a temp view
curated_df = spark.read.parquet(CURATED_PATH)
curated_df.createOrReplaceTempView("sensor_curated")

print("Schema of curated table:")
curated_df.printSchema()

```

createOrReplaceTempView("sensor_curated") registers the DataFrame as a virtual SQL table available for the duration of the SparkSession. No data is copied – the view is just a reference. You can now query it with **spark.sql()** using standard SQL syntax.

5.5.2 4b – Query 1: Basic Partition Pruning

```

# This query uses partition pruning on sensor_type and day
result1 = spark.sql("""
    SELECT
        sensor_type,
        day,
        ROUND(AVG(value), 2)    AS avg_value,
        ROUND(MIN(value), 2)    AS min_value,

```

```

        ROUND(MAX(value), 2)    AS max_value,
        COUNT(*)                AS total_records
    FROM sensor_curated
    WHERE sensor_type = 'temperature'
        AND year = 2024
        AND month = 1
    GROUP BY sensor_type, day
    ORDER BY day
    """)

result1.show()

```

WHERE sensor_type = 'temperature': Spark reads *only* the sensor_type=temperature/ directory – all humidity and pressure files are skipped entirely without even opening them.

AND year = 2024 AND month = 1: further prunes to only year=2024/month=01/. Spark combines all filter conditions before accessing any file.

Run `result1.explain(True)` to see the query plan and confirm “PartitionFilters” appear in the Scan node.

5.5.3 4c – Query 2: Anomaly Analysis

```

result2 = spark.sql("""
    SELECT
        sensor_type,
        COUNT(*)                AS total,
        SUM(CAST(is_anomaly AS INT)) AS anomalies,
        ROUND(
            100.0 * SUM(CAST(is_anomaly AS INT)) / COUNT(*),
            2
        )                        AS anomaly_pct
    FROM sensor_curated
    GROUP BY sensor_type
    ORDER BY anomaly_pct DESC
    """)

result2.show()

```

5.5.4 4d – Measure the Pruning Benefit

```

# Full table scan (no partition filter) -- measures baseline
import time

start = time.time()
spark.sql(
    "SELECT COUNT(*) FROM sensor_curated"
).collect()
full_scan_time = time.time() - start

# Pruned scan (temperature only)

```

```
start = time.time()
spark.sql("""
    SELECT COUNT(*)
    FROM sensor_curated
    WHERE sensor_type = 'temperature'
""").collect()
pruned_time = time.time() - start

print(f"Full scan time      : {full_scan_time:.2f}s")
print(f"Pruned scan time   : {pruned_time:.2f}s")
print(f"Speedup              : {full_scan_time/pruned_time:.1f}x")
```

With a 3-sensor dataset, the speedup is roughly $3\times$ (one partition read instead of three). In real data lakes with hundreds of partitions (e.g., 365 days $\times$ 100 sensor types = 36,500 partitions), partition pruning can reduce scan time by **100–1000 $\times$** .

Reflection Questions

Answer before Session 5:

1. You have a dataset with columns: `user_id` (10M distinct values), `country` (50 values), `event_date`, `event_type` (8 values). Which columns would you choose for partitioning and why?
2. Explain why the raw zone uses *ingestion time* for its directory partitioning, while the curated zone uses *event time*. In what situation could a gap between the two times indicate a problem?
3. What is the “small file problem” and what are its two main consequences on query performance?
4. A new field `firmware_version` is added to the sensor payload starting January 20. Old Parquet files (before Jan 20) do not have this field. What happens when Spark reads both old and new files together?
5. You delete the curated zone checkpoint and restart the streaming pipeline. What happens to the raw zone checkpoint? Does data get duplicated in the curated zone?

6. Summary & Key Takeaways

📌 What We Covered Today

1. A **data lake** stores all data in raw form using cheap object storage; schema is applied at read time (schema-on-read).
2. The **three-zone architecture** (Raw → Curated → Consumption) prevents data swamps and enforces progressive quality improvement.
3. **Parquet** is the columnar format of choice for data lakes: column pruning, predicate pushdown, and excellent compression. ORC is preferred in Hive environments; Avro for streaming writes (Kafka).
4. **Partition pruning** is the single most impactful query optimisation: only matching directories are read. Always partition on frequently filtered, low-cardinality columns.
5. **Hive-style partitioning** (key=value directories) is recognised automatically by Spark, Hive, Athena, and Presto.
6. The **small file problem** degrades both write and read performance; mitigate with compaction, longer trigger intervals, or table formats like Delta Lake.
7. In the lab, we built a three-zone lake: Kafka → Raw (JSON) → Curated (Parquet, partitioned) → Consumption (aggregated) with Spark SQL queries.

6.1 Vocabulary Checklist

- | | |
|-------------------------------------|-------------------------------|
| ✓ Data lake / data swamp | ✓ Predicate pushdown |
| ✓ Data warehouse vs lakehouse | ✓ Column pruning |
| ✓ Schema-on-read vs schema-on-write | ✓ ORC / Avro |
| ✓ Raw zone (bronze) | ✓ Hive-style partitioning |
| ✓ Curated zone (silver) | ✓ Partition pruning |
| ✓ Consumption zone (gold) | ✓ Small file problem |
| ✓ Medallion architecture | ✓ Compaction |
| ✓ Row-oriented vs columnar storage | ✓ Data catalogue / lineage |
| ✓ Parquet row group / column chunk | ✓ Delta Lake / Apache Iceberg |

6.2 Further Reading

- Chambers, B. & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly. Chapter 9 (Data Sources) and Chapter 19 (Performance Tuning).

- Apache Parquet specification: parquet.apache.org/docs/file-format
- Delta Lake documentation: docs.delta.io
- Apache Iceberg documentation: iceberg.apache.org/docs
- Inmon, W.H. (2002). *Building the Data Warehouse*, 4th ed. Wiley. (foundational DW concepts, for contrast with lakes)

6.3 Preview: Session 5

Next session we expose our data lake through APIs:

- REST API principles and HTTP verbs.
- JSON data exchange and status codes.
- Lab: Build a Flask-based REST API that queries the Parquet data lake and returns sensor statistics.

Appendix A – Complete datalake_pipeline.py

```

"""
Session 4 -- Data Lake Pipeline
Kafka -> Raw Zone (JSON) -> Curated Zone (Parquet)
"""

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, from_json, to_timestamp, expr,
    year, month, dayofmonth, hour,
)
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, LongType, BooleanType,
)

LAKE_ROOT      = "/tmp/datalake"
RAW_PATH       =
    f"{LAKE_ROOT}/raw/source=kafka/topic=sensor-events"
CURATED_PATH   = f"{LAKE_ROOT}/curated/domain=iot"
CKPT_RAW       = "/tmp/datalake-ckpt/raw"
CKPT_CUR       = "/tmp/datalake-ckpt/curated"
KAFKA_BROKERS  = "localhost:9092,localhost:9094,localhost:9096"
TOPIC          = "sensor-events"

SENSOR_SCHEMA = StructType([
    StructField("sensor",    StringType(), False),
    StructField("value",     DoubleType(), False),
    StructField("unit",      StringType(), True),
    StructField("timestamp", LongType(),    False),
    StructField("source",    StringType(), True),
    StructField("anomaly",   BooleanType(), True),
])

spark = SparkSession.builder \
    .appName("DataLakePipeline") \
    .master("local[*]") \
    .config(
        "spark.jars.packages",
        "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3"
    ) \
    .config("spark.sql.shuffle.partitions", "3") \
    .getOrCreate()
spark.sparkContext.setLogLevel("WARN")

raw_kafka = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
    .option("subscribe", TOPIC) \
    .option("startingOffsets", "earliest") \

```

```

.option("failOnDataLoss", "false") \
.load()

# Raw zone: land JSON as-is, partitioned by ingestion time
raw_for_lake = raw_kafka.select(
    col("value").cast("string").alias("raw_json"),
    col("partition").alias("kafka_partition"),
    col("offset").alias("kafka_offset"),
    col("timestamp").alias("ingestion_ts"),
    year(col("timestamp")).alias("year"),
    month(col("timestamp")).alias("month"),
    dayofmonth(col("timestamp")).alias("day"),
    hour(col("timestamp")).alias("hour"),
)

query_raw = raw_for_lake.writeStream \
    .outputMode("append").format("json") \
    .option("path", RAW_PATH) \
    .option("checkpointLocation", CKPT_RAW) \
    .partitionBy("year", "month", "day", "hour") \
    .trigger(processingTime="30 seconds").start()

# Curated zone: parse JSON, validate, partition by event time
parsed = raw_kafka.select(
    from_json(col("value").cast("string"),
              SENSOR_SCHEMA).alias("d"),
    col("partition"), col("offset"),
).select(
    col("d.sensor").alias("sensor_type"),
    col("d.value"), col("d.unit"),
    col("d.anomaly").alias("is_anomaly"),
    to_timestamp(expr("d.timestamp / 1000")).alias("event_time"),
    col("partition"), col("offset"),
).filter(col("sensor_type").isNotNull()) \
.filter(col("value").isNotNull()) \
.filter(col("value").between(-100, 2000))

curated = parsed \
    .withColumn("year", year(col("event_time"))) \
    .withColumn("month", month(col("event_time"))) \
    .withColumn("day", dayofmonth(col("event_time")))

query_curated = curated.writeStream \
    .outputMode("append").format("parquet") \
    .option("path", CURATED_PATH) \
    .option("checkpointLocation", CKPT_CUR) \
    .option("compression", "snappy") \
    .partitionBy("sensor_type", "year", "month", "day") \
    .trigger(processingTime="30 seconds").start()

print("Pipeline running. Ctrl+C to stop.")

```

```
try:
    spark.streams.awaitAnyTermination()
except KeyboardInterrupt:
    for q in spark.streams.active:
        q.stop()
spark.stop()
```

Appendix B – Glossary

lightbg Term	Definition
Avro	Row-oriented binary format with embedded schema; optimised for streaming writes (Kafka).
Column pruning	Reading only the requested columns from a Parquet/ORC file, skipping all others.
Compaction	Merging many small files into fewer, larger files to improve query performance.
Consumption zone	Gold layer: aggregated, business-ready datasets built from the curated zone.
Curated zone	Silver layer: cleaned, validated, Parquet-formatted data partitioned by event time.
Data catalogue	Inventory of all datasets with schema, owner, freshness, and lineage metadata.
Data lake	Centralised store for raw data in native format, using cheap object storage.
Data lakehouse	Data lake with ACID transactions and schema enforcement (Delta Lake, Iceberg).
Data swamp	A data lake without governance: data is ingested but undiscoverable and untrustworthy.
Hive partitioning	key=value directory naming automatically recognised by Spark, Hive, and Presto.
ORC	Optimised Row Columnar: columnar format for Hive with built-in indexes and ACID support.
Parquet	Columnar binary format with row group statistics; default for Spark and modern cloud analytics.
Partition pruning	Skipping directories that do not match query filters, reducing data scanned.
Predicate pushdown	Evaluating filter conditions at the storage layer (row group statistics) to skip reading data.
Raw zone	Bronze layer: immutable copy of source data in original format.
Row group	Horizontal slice of a Parquet file containing column chunks and statistics.
Schema-on-read	Schema is applied when data is queried, not when it is written.
Small file problem	Performance degradation caused by thousands of tiny files overwhelming metadata operations.